Contents

Decision Log

Format: each entry is “considered X, chose Y, because Z.” Latest decisions at top.

Hide empty rows in comparison table

Considered keeping every row (Rating, Top features, Specs, Action) with – placeholders for missing data, vs. hiding rows where no product has data. Chose to hide. Because rows full of dashes hurt the perceived quality of the agent more than the missing axis does — the buyer reads “—” as the agent’s failure, not the catalog’s.

Filter low-rated products before ranking (default sort)

Considered ranking purely by Shopify’s relevance order, vs. dropping products with rating < 3.0 before ranking. Chose to filter, with a fallback to unfiltered if fewer than 3 products survive. Because a 1-star product showing up first looked broken in user testing, even when relevance-correct. The fallback prevents empty result sets in sparse categories.

Sync messages from DB after stream **`onFinish`**

Considered relying on the live in-memory part state from `useChat`, vs. force-replacing messages with the DB-fresh snapshot when the stream finishes. Chose force-replace. Because a tool part occasionally stayed in `state="streaming"` on the client even though the backend had committed `state="output-available"`, leaving products invisible until navigation. The DB is the source of truth post-stream.

Strip markdown pipe tables and JSON envelopes from assistant text

Considered tightening the system prompt to forbid both, vs. tightening the prompt AND adding a frontend stripper. Chose both. Because the system prompt already forbids markdown tables and instructs the JSON envelope shape, but free-tier models still leak. The frontend guard is the only durable defense.

Only render the last compareProducts table in a message

Considered showing every `tool-compareProducts` invocation, vs. only the last one. Chose the last one. Because the model sometimes runs compareProducts twice (different subsets) within a single turn, and showing both stacked looked like a bug.

Catalog search not cached

Considered caching `search_catalog` responses in Redis, vs. obeying Shopify's "do not cache search results or catalog images" policy. Chose policy compliance. Because catalog freshness is the entire point, and a UCP profile signed by us implicitly accepts the rules of the road. `cache: "no-store"` on every MCP call. Catalog images render directly from merchant URLs, not through a Next/Image optimizer.

Recommendation feed is non-AI

Considered making the homepage an AI loop ("here is what's hot today"), vs. a deterministic feed backed by Redis cache. Chose deterministic. Because the agent is rate-limited and the feed should be free to browse without spending a free-tier token. Cache TTLs: 1h for recommendations, 5m for search. Daily-rotated category bucket means most users hit a cached response.

Clarify only when missing info changes the answer

Considered forcing clarifyIntent at the start of every shopping turn, vs. clarifying only when a missing constraint materially changes the product set. Chose the latter. Because forced clarification on every turn made the chat feel like a form. Tradeoff accepted: the agent sometimes picks one product class when two were plausible.

Stop the stream on **`clarifyIntent`**

Considered letting the model continue after asking a clarification, vs. treating clarifyIntent as terminal for the turn. Chose terminal (`stopWhen: hasToolCall("clarifyIntent")`). Because mixing a question and a product carousel in the same turn confused the UI and the buyer; the chips should be the only thing they react to.

Per-chat seeded variation

Considered identical outputs for identical queries, vs. seeded variation keyed on `chatId`. Chose seeded variation across: - system prompt personality hint, - search-result tail shuffle, - suggested-action chips, - greeting copy.

Because two judges hitting the same demo should see different conversational tone without different facts. Within a single chat, outputs stay stable across reloads.

Per-request **`SearchMemo`**, not DB-persisted

Considered persisting the last query/filters/result IDs in the DB, vs. reconstructing on every request from message history. Chose reconstruction. Because adding a DB write per turn for cache state is operationally heavier than rehydrating from existing message parts, and a chat that has no message history has no memo to rehydrate.

Generative UI **>** model-emitted markdown

Considered letting the model render product info as markdown, vs. forcing native UI components keyed on tool outputs. Chose native components. Because the model cannot fabricate a tool result, so the UI is grounded by construction. The prompt explicitly forbids markdown tables and headings.

Free-tier OpenRouter primary, DeepSeek direct optional

Considered paid OpenAI / Anthropic for reliability, vs. free OpenRouter with fallback chain **+** direct DeepSeek. Chose free with fallbacks. Because the hackathon submission should be reproducible without committing paid keys. Tradeoff: occasional rate limit during the demo, mitigated by the fallback chain.

Disable DeepSeek thinking mode at the API layer

Considered enabling DeepSeek's native thinking mode, vs. disabling it for now. Chose disabling. Because thinking mode requires `reasoning_content` to be preserved and replayed across follow-up tool turns, and the current AI SDK / UI-message persistence path drops the provider-specific field. Disabling keeps direct DeepSeek usable with tools.

Guest auth as the dominant CTA

Considered making login/register the only entry, vs. exposing "Continue as guest" as the primary action on both pages. Chose guest. Because demo time is a strict budget — judges should not sign up to evaluate. Guest user rows are real Postgres rows so the same persistence path works.

Backend gated by **`BACKEND_INTERNAL_SECRET`**

Considered open backend endpoints, vs. requiring an internal-secret header on every frontend → backend call. Chose the secret. Because the catalog data is not user-scoped but the LLM cost is real; the secret keeps random scrapers from hammering the agent surface.

Single-file backend **`server.ts`**

Considered modularizing routes into separate files, vs. one Hono file. Chose one file for now. Because the surface is small (chat, messages, votes, feed) and one file keeps the streaming + persistence flow visible end-to-end. Split when it crosses ~800 lines or when route count > ~10.

Freshness audit as a separate tool

Considered baking generation detection into `searchProducts`, vs. exposing a dedicated `assessProductFreshness` tool. Chose the dedicated tool. Because the search response gains a `freshnessHint` field telling the model "you may want to run the audit", which makes the decision visible to the model and to the buyer (via the GenerationVerdictCard) rather than hidden inside ranking logic.

Web search via Playwright with provider chain

Considered using a commercial search API (Brave, Serper, Tavily), vs. Playwright over Bing → DuckDuckGo → Google. Chose Playwright. Because the hackathon submission should run on a laptop with no SaaS account. Tradeoff: slower, occasional CAPTCHA on Google from datacenter IPs — mitigated by the provider chain and an optional `SEARCH_PROXY_URL`.

Track-agnostic agent registry from day one

Considered hard-coding Track 1 logic in the server, vs. designing a registry that maps `agentId` → `AgentDefinition`. Chose the registry. Because Tracks 2–5 should be addable without surgery, and the registry shape is cheap. Tracks 2–5 are currently stubs.

Dropped own Shopify dev store + CSV import path

Considered seeding a Shopify development store with a custom electronics CSV, vs. using the public Catalog MCP. Chose Catalog MCP. Because Catalog MCP gives cross-merchant data, real `checkoutUrl`s per variant, and the literal Track 1 framing ("across Shopify merchants"). The CSV pipeline still exists in `product_extractor/` for reference but is not the runtime.

Dropped artifact tools from the chatbot template

Considered keeping `createDocument`, `editDocument`, `getWeather` etc. wired in, vs. unwiring from the agent. Chose to unwire. Because they pulled the demo toward "generic chatbot" and away from Track 1. The component code is left dormant for type stability; cleanup deferred.

Dropped the custom shopping dashboard shell

Considered keeping the static recommendation grid + saved-items dashboard, vs. removing it. Chose to remove. Because it made the app look like a fake ecommerce store rather than a Vercel-style chatbot, and none of the static data was grounded in the real Catalog MCP. Shopping-specific UI now lives only inside chat messages and the live feed.